Output locality and recursive schemes

1. Output Strictly Local functions

Output Strictly Local (OSL) functions are string functions whose behavior is determined by a bounded suffix of the output generated so far. They are particularly useful in the study of phonological processes in which an output segment can influence the production of later output segments.

Example: iterative nasal spreading. In Johore Malay, nasality spreads rightward through vowels and glides. Chandlee, Eyraud & Heinz (2015) give the following form, attributed to Onn (1980):

$$/\text{pəŋawasan}/ \longmapsto [\text{pəŋãw̃ãsan}] \qquad \text{'supervision'}.$$

The first vowel after [ŋ] is nasalized, the glide after that vowel, and the second vowel after the glide, with spreading stopping at [s].

Each newly nasalized segment provides the context for nasalization of the next segment. Although the original trigger can be arbitrarily far from a target in the *input*, the relevant *output* context remains adjacent. At the blocker, spreading stops. This illustrates left 2-OSL behavior, since only one neighboring output symbol is needed.

Direction matters. Left-OSL processes from left to right; right-OSL imposes the corresponding restriction when processing proceeds from right to left. In the Tianjin low-tone mapping illustrated by Oakden (2021), L becomes R before an output L:

$$LLLL \longmapsto RLRL \qquad \text{(process from right to left)}.$$

The final L remains unchanged; its left neighbor becomes R; the next L remains L because its output neighbor is now R. This mapping is right 2-OSL.

The formal condition

A subsequential function $f : \Sigma^* \to \Gamma^*$ is *left k-OSL* if, for all input prefixes $w_1, w_2 \in \Sigma^*$,

$$\text{Suff}^{k-1}(\mathring{f}(w_1)) = \text{Suff}^{k-1}(\mathring{f}(w_2)) \quad \Longrightarrow \quad f_{w_1} = f_{w_2}.$$

Intuitively, once two computations have produced output ending in the same last $k-1$ symbols, they must respond in the same way to every possible continuation of the input. Their earlier histories may differ, but those differences cannot affect what remains to be produced. This is what equality of the *tail functions* expresses. For nasal spreading, for example, what matters is whether the most recent output segment is nasal, rather than how far back the original trigger occurred.

The technical definitions of committed output, suffixes, and tail functions appear at the end of the handout.

2. Boolean monadic recursive schemes

BMRS provide a logical framework for defining string functions through recursively defined Boolean predicates. Think of a string as a set of positions, each labeled with a symbol. The predecessor function p returns the preceding position; the successor function s returns the following position.

A BMRS is a finite collection of conditional definitions that may refer to one another. The functions are **Boolean** because they return truth values, and **monadic** because each takes only one position

as its argument. **Recursion** allows a definition to call itself or another defined function. (Bhaskar et al. 2020; Chandlee & Jardine 2021.)

BMRS Ingredients

BMRS terms have two types: an *index term* denotes a string position, and a *Boolean term* evaluates to a truth value. A program consists of finitely many equations $f_j(x) = t_j$, with a Boolean right-hand side for each recursively defined function f_j. The following are the available building blocks.

1. **A position variable:** x is an index term.
2. **Navigation:** $p(t)$ and $s(t)$ are index terms when t is an index term. They denote its predecessor and successor. Navigation can be nested, as in $p(p(x))$.
3. **Truth values:** $\top$ and $\bot$ are Boolean terms, meaning true and false.
4. **Input-label tests:** $\sigma(t)$ is a Boolean term asking whether position t has input label σ. The input alphabet includes boundary labels, so $\rtimes(t)$ can test for the left boundary. These predicates are supplied by the input structure, not defined recursively by the program.
5. **Recursive calls:** $f_j(t)$ is a Boolean term when t is an index term. Its value is determined by the equation defining f_j, which may call itself or other functions in the same finite program.
6. **Equality tests:** $t_1 = t_2$ is Boolean when the two terms have the same type.
7. **Conditionals:** `if` b `then` t_1 `else` t_2 first evaluates the Boolean test b and selects the appropriate branch. The branches must have the same type; the conditional inherits that type.

Boolean connectives such as $\neg$, $\wedge$, and $\vee$ are convenient abbreviations expressible using conditionals. For example, $b_1 \wedge b_2$ abbreviates `if` b_1 `then` b_2 `else` $\bot$.

General recursive functions versus output predicates. All the recursively defined functions are Boolean and monadic. For a string transduction, we designate some of them as *output predicates*, written γ for $\gamma \in \Gamma$, where Γ is the output alphabet. The equation for γ determines whether output label γ holds at a position. Any remaining recursive functions, written $f_1, \ldots, f_r$, are *auxiliary*: they can contribute to the computation, but their truth values do not themselves supply output labels. Output predicates are therefore recursive functions with a designated output role, rather than a different type of function.

For example, an auxiliary function may record whether an a has occurred at or before the current position:

$$f_1(x) = \texttt{if } \rtimes_i(x) \texttt{ then } \bot \\ \texttt{else if } a_i(x) \texttt{ then } \top \texttt{ else } f_1(p(x)).$$

The boundary test stops the recursion. An output equation could consult $f_1(p(x))$ to use this information, even when it is not visible in the preceding output label. A general BMRS can also inspect another position's input directly, as in $a_i(p(x))$. Both possibilities matter when we later impose restrictions.

From truth values to an output string. With one copy of each input position, $\gamma(x) = \top$ assigns output label γ to that position. In the spreading example below, exactly one designated output predicate is true at each ordinary position, and reading those labels in position order gives

the output string. More generally, output predicates have the form $\gamma^e(x)$ for a copy index $e \in C$: a position can contribute several symbols, or none. At each occupied copy, exactly one output label must hold for the output to be well formed. The labels are read in input-position order and, within each position, in copy order.

A logical description of spreading

We can now give a BMRS description of a schematic version of the Johore Malay pattern. Let N represent a nasal segment, T an oral target vowel or glide, and B a blocking consonant. In the output, N represents both nasal consonants and nasalized targets. These symbols represent segment classes, abstracting away from individual segment identities. The substring /ŋawas/ corresponds to $NTTTB$; the schematic example below adds a target after the blocker to show that it remains oral.

$$\text{NTTTBT} \longmapsto \text{NNNNBT}.$$

Here the output alphabet is $\Gamma = \{N, T, B\}$. Its designated output predicates are $\gamma_N, \gamma_T, \gamma_B$: for example, $\gamma_N(x)$ asks whether position x is nasal in the output. The input predicates N_i, T_i, B_i test its input class. There are no auxiliary recursive functions in this example.

$$\begin{aligned}
\gamma_N(x) &= \texttt{if } N_i(x) \texttt{ then } \top \\
&\qquad \texttt{else if } T_i(x) \texttt{ then } \gamma_N(p(x)) \texttt{ else } \bot, \\
\gamma_B(x) &= B_i(x), \\
\gamma_T(x) &= \texttt{if } T_i(x) \texttt{ then} \\
&\qquad (\texttt{if } \gamma_N(x) \texttt{ then } \bot \texttt{ else } \top) \texttt{ else } \bot.
\end{aligned}$$

The equation for γ_N preserves input nasals and captures spreading: a target is nasalized if its predecessor is nasal in the output. At the first target, γ_N is true because the preceding output is a nasal consonant. At subsequent targets, it is true because the preceding output is a nasalized target. At the blocker, only γ_B is true, so the following target remains oral. All ordinary input predicates, and hence these output predicates, are false at the boundaries.

Which restrictions connect BMRS to OSL?

Restriction	What it permits
BMRS^p	Predecessor p, but no successor s; reference to current or earlier positions.
BMRS^s	Successor s, but no predecessor p; reference to current or later positions.
Output restriction	Input-symbol tests only at the current position: σ(x), never σ(p(x)) or σ(s(x)).
No auxiliary recursion	The only recursive predicates are output predicates. Information about other positions must come through their output labels.

The last condition is essential: auxiliary predicates could otherwise carry information that is absent from the output. **OR-BMRS** includes the output restriction together with this ban on auxiliary recursive predicates.

3. Why the output restriction is not enough

Oakden (2021) conjectured that output-restricted BMRS characterize the OSL functions, with predecessor-only systems for left-OSL and successor-only systems for right-OSL. The idea was to

test only the current input symbol and use recursive calls for context. The examples below show a further problem: even without auxiliary predicates, two histories can produce the same visible output and still respond differently to the same continuation. The central problem occurs in the direction from OR-BMRS-definability to OSL. A BMRS predicate inspects copies of *input positions*; an OSL computation retains a suffix of the *visible output string*. These need not provide the same information.

Copies and blocks. With a copy set $C = \{1, \dots, m\}$, each input position can contribute up to m output symbols, one per occupied copy. Copies are ordered within each input position, and these blocks are concatenated in input order. Flattening erases copy identities and the divisions between blocks.

Hidden copy identities

Let $\Sigma = \Gamma = \{a, b, c\}$ and $C = \{1, 2\}$. Here $a(x)$, $b(x)$, and $c(x)$ are input predicates, while $\gamma^e(x)$ is the output predicate for symbol γ in copy e. Define

$$\mathrm{Empty}(y) := \bigwedge_{\substack{\gamma \in \Gamma \\ e \in C}} \neg \gamma^e(y).$$

Empty is an abbreviation for a finite conjunction, not an auxiliary recursive predicate. The system is:

$$\begin{aligned}
a^1(x) &= \big(a(x) \wedge \mathrm{Empty}(p(x))\big) \vee \big(c(x) \wedge a^1(p(x))\big),\\
a^2(x) &= \big(b(x) \wedge \mathrm{Empty}(p(x))\big) \vee \big(c(x) \wedge a^2(p(x))\big),\\
b^1(x) &= a(x) \wedge a^1(p(x)),\\
c^1(x) &= a(x) \wedge a^2(p(x)),\\
b^2(x) &= \bot,\\
c^2(x) &= \bot.
\end{aligned}$$

This system is output-restricted because every input-symbol predicate is evaluated only at x, and it belongs to BMRS^p because it uses no successor terms.

The following diagram is taken from the thesis discussion. A dash means that the corresponding copy produces no symbol. The systems use only current input tests and predecessor output predicates.

Input ac^na	a	c	$\cdots$	c	a
copy 1	a	a	$\cdots$	a	b
copy 2	-	-	$\cdots$	-	-
output	a	a	$\cdots$	a	b
Input bc^na	b	c	$\cdots$	c	a
copy 1	-	-	$\cdots$	-	c
copy 2	a	a	$\cdots$	a	-
output	a	a	$\cdots$	a	c

Before the final a, both computations have produced a^{n+1}. But one carries this output in copy 1 and the other in copy 2. The shared continuation a produces b in the first computation and c in the second. Since n is unbounded, no fixed visible output suffix determines the continuation behavior.

4. What positionwise length preservation buys us

With a **singleton copy set** and **exactly one output symbol at each ordinary input position**, copy indices cannot act as additional labels, and each output block contains exactly one symbol. Boundary positions produce no output. There is consequently a one-to-one correspondence between ordinary input positions and output positions. Let b_i be the output symbol produced at input position x_i. Then the output symbol j places earlier, b_{i-j}, was produced at input position $p^j(x_i) = x_{i-j}$. The last k-1 output symbols can therefore be matched directly with the last k-1 input positions, allowing the output suffix to determine the predicate values available to the system.

The correspondence

Under the positionwise length-preserving restriction:
left-OSL mappings $\leftrightarrow$ mappings definable by OR-BMRS in BMRS^p.
The symmetric correspondence holds for right-OSL and BMRS^s.

OSL $\rightarrow$ OR-BMRS. Recover each transducer state from its bounded preceding output history, then define each output predicate by checking that history together with the current input symbol. For example, suppose a left 3-OSL transducer produces c when the preceding output ends in ab and the current input is t. The equation for $\gamma_c(x)$ includes the case

$$t_i(x) \ \wedge \ \gamma_a(p^2(x)) \ \wedge \ \gamma_b(p(x)).$$

The last two tests recover the relevant history directly from the preceding output predicates, so no auxiliary state predicate is needed.

OR-BMRS $\rightarrow$ OSL. A finite program has a maximum predecessor depth d. Positionwise alignment makes the preceding d output symbols sufficient to determine all predecessor predicate values available at the current position. If two histories have the same output suffix of length at most d and read the same next input symbol, they produce the same next output symbol. Appending that symbol to both outputs keeps their suffixes equal, so the argument repeats at every step of any shared continuation. Thus the two histories have the same tail function, giving the bound $k = d + 1$.

The significance is that the correspondence follows from a simple restriction on BMRS transductions. Positionwise length preservation removes the additional information carried by copy identities and block boundaries without explicitly requiring the system to behave like an OSL transducer.

Next steps / questions for discussion

Beyond length preservation. Epenthesis and deletion break the direct correspondence between input and output positions. What restrictions would let a bounded visible output suffix recover the information used by recursion?

Characterize the larger class. If unrestricted OR-BMRS can retain information in copies and blocks, which mappings does this class define? (It appears to be the subsequential functions!)

Learning. Can a learner infer logical descriptions combining current-input tests with bounded output context?

Selected references

Chandlee, Eyraud & Heinz (2015). Output Strictly Local Functions. *MoL*, 112–125.
Bhaskar et al. (2020). Boolean Monadic Recursive Schemes as a Logical Characterization of the Subsequential Functions. *LATA*, 157–169.
Chandlee & Jardine (2021). Computational Universals in Linguistic Theory: Using Recursive Programs for Phonological Analysis. *Language* 97(3), 485–519.
Oakden (2021). *Modeling Phonological Interactions Using Recursive Schemes.* Rutgers dissertation, §3.4.3.

Optional: the formal OSL condition and tail functions

Let $f : \Sigma^* \to \Gamma^*$ be a subsequential function. We want to describe what remains to be computed after reading an input prefix w. The definition separates the output already determined by w from the output that still depends on its continuation.

1. The output already committed. Write

$$\mathring{f}(w) = \mathrm{lcp}\{f(wz) : z \in \Sigma^*\},$$

where lcp means longest common prefix. This is the part of the output shared by every possible completion of w: it can be committed without knowing which continuation z comes next. This definition concerns the function itself, rather than the timing of output in a particular transducer.

2. The tail function. The *tail function* f_w maps a continuation z to the output remaining after the committed prefix is removed. In other words, it is defined by

$$f(wz) = \mathring{f}(w)\, f_w(z).$$

The subscript w records the input history. The argument z is the new material still to be processed. Thus f_w describes the entire remaining input–output behavior after w, not just the next output symbol.

Two prefixes w_1 and w_2 have the same tail function if

$$f_{w_1} = f_{w_2} \quad \iff \quad f_{w_1}(z) = f_{w_2}(z) \text{ for every } z \in \Sigma^*.$$

Their already committed outputs can differ; what must agree is the remaining output on every shared continuation.

3. Retaining only a bounded output suffix. Let $\mathrm{Suff}^{k-1}(u)$ be the last at most $k-1$ symbols of u (all of u if it is shorter). The function f is *left k-OSL* if, for all input prefixes w_1, w_2,

$$\mathrm{Suff}^{k-1}(\mathring{f}(w_1)) = \mathrm{Suff}^{k-1}(\mathring{f}(w_2)) \quad \Longrightarrow \quad f_{w_1} = f_{w_2}.$$

Read this as: *if two histories have the same bounded suffix of committed output, then they must have the same tail function.* No other distinction between those histories may affect the remaining computation. The implication does not require different suffixes to have different tail functions.

For the schematic nasal-spreading mapping, histories whose committed outputs end in N allow the same continuation behavior, regardless of how far back the original nasal trigger occurred. A following target is nasalized abd a blocker stops spreading. This is why retaining one output symbol can suffice even though the effect extends over an arbitrarily long string. The formal condition requires this agreement for *every* continuation, not only for a single following target. (Chandlee, Eyraud & Heinz 2015.)

Left packing still hides block boundaries

Suppose that whenever copy e is occupied, all earlier copies are occupied too. This *left-packing* condition fixes the copy assignment within a block, but the flattened output still does not identify block boundaries.

Again let $\Sigma = \Gamma = \{a, b, c\}$ and $C = \{1, 2\}$, and use the same abbreviation Empty. Define:

$$\begin{aligned}
a^1(x) &= \big((a(x) \vee b(x)) \wedge \mathrm{Empty}(p(x))\big) \\
&\quad \vee \big(c(x) \wedge (b^1(p(x)) \vee b^2(p(x)))\big), \\
b^1(x) &= \big(c(x) \wedge a^1(p(x)) \wedge \neg b^2(p(x))\big) \\
&\quad \vee \big(a(x) \wedge b^1(p(x))\big), \\
b^2(x) &= \big(a(x) \wedge \mathrm{Empty}(p(x))\big) \\
&\quad \vee \big(c(x) \wedge b^2(p(x))\big), \\
c^1(x) &= a(x) \wedge b^2(p(x)), \\
a^2(x) &= \bot, \\
c^2(x) &= \bot.
\end{aligned}$$

This system is output-restricted and left packed: whenever $b^2(x)$ is true, $a^1(x)$ is also true.

Input aa	a	a	Input bca	b	c	a
copy 1	a	c	copy 1	a	b	b
copy 2	b	-	copy 2	-	-	-
block	ab	c	block	a	b	b

Flattened outputs: ab | c = abc and a | b | b = abb.

The common prefix ab is one block in the first computation and two blocks in the second. More generally, prefixes ac^n and bc^{2n+1} both produce $(ab)^{n+1}$, but a shared continuation a produces c versus b. Left packing therefore does not restore OSL locality.